

机器学习分类算法在量化交易中的应用

——基于乳腺癌数据集与股票收益数据的实证分析

姓 名：陈柳萱

学 号：2500015450

课 程：量化投资策略（TASK5）

提交日期：2026-07-18

GitHub 交互网址：<https://cccl111xxx.github.io/quant/hw5/>

一、引言

机器学习算法在量化交易领域的应用日益广泛。它能够帮助交易者从海量行情与财务数据中挖掘有价值的信息，构建更精准的预测模型，从而优化交易决策。与传统的线性回归或技术指标择时相比，分类模型能够直接输出离散的交易信号（如买入/卖出、涨/跌），与策略执行天然契合。

本报告首先介绍三类基础分类算法——逻辑回归、决策树与随机森林的原理与适用场景；其次说明混淆矩阵、ROC 曲线与 AUC 等核心评价指标；最后以 sklearn 乳腺癌数据集与贵州茅台日行情构造的次日涨跌数据集进行实证，训练并对比三类模型，给出 ROC 曲线与混淆矩阵，并分析实验结果。

二、分类机器学习算法

（一）逻辑回归

逻辑回归（Logistic Regression）是一种广义线性分类模型。它先对输入特征做线性组合 $z = \mathbf{w}'\mathbf{x} + b$ ，再通过 Sigmoid 函数 $s(z) = 1 / (1 + e^{-z})$ 将结果映射到 (0, 1) 区间，得到样本属于正类的概率。当概率高于阈值（通常取 0.5）时判为正类，否则判为负类。

逻辑回归的优点是可解释性强、训练速度快、输出具有概率意义；缺点是只能刻画特征与标签之间的线性决策边界，对复杂非线性关系的表达力有限。

（二）决策树

决策树（Decision Tree）通过递归地选择最优特征与切分点，将样本空间划分成若干矩形区域，每个区域对应一个类别预测。常用指标包括信息增益（ID3）、信息增益率（C4.5）与基尼指数（CART）。

决策树直观易懂，能够自动捕捉特征非线性交互，但单棵决策树容易过拟合，泛化能力较弱。

（三）随机森林

随机森林（Random Forest）是一种集成学习方法。它通过自助采样（Bagging）构建多棵决策树，每棵树在随机选取的特征子集上训练，最终以投票或平均概率的方式输出预测结果。该策略显著降低了单棵决策树的方差，提高了模型稳定性与预测精度。

三、模型评价指标

（一）混淆矩阵

混淆矩阵（Confusion Matrix）是二分类问题的基础评估工具。它将模型预测结果与真实标签交叉，得到真正例（TP）、假正例（FP）、真负例（TN）与假负例（FN）四个格子。由其可进一步计算准确率、精确率、召回率与 F1 值等常用指标。

（二）ROC 曲线

ROC 曲线（Receiver Operating Characteristic Curve）以假正例率（FPR = FP / (FP + TN)）为横轴，以真正例率（TPR = TP / (TP + FN)）为纵轴，通过改变分类阈值绘制得到。曲线越靠近左上角，说明模型在控制误判的同时能识别出更多的正例，分类性能越好。

（三）AUC

AUC（Area Under Curve）是 ROC 曲线下的面积，取值范围为 [0, 1]。AUC 可理解为随机抽取一个正例与一个负例，模型给正例打分更高的概率。AUC 越接近 1，模型区分能力越强；AUC 为 0.5 时相当于随机猜测。

四、实证分析

（一）数据来源与变量说明

本实验使用两个数据集。数据集一为 sklearn 内置的乳腺癌数据集 (load_breast_cancer)，包含 569 个样本、30 个医学特征，标签为 0（恶性）或 1（良性），属于标准的 0/1 二分类问题。

数据集二为量化场景数据。读取贵州茅台（600519.SH）日行情，计算收益率、动量、波动率、成交量变化、均线偏离等技术指标作为特征，标签定义为次日收益率是否大于 0，即 1 表示上涨、0 表示下跌，共 825 条有效样本。

（二）实验设计

对两个数据集均采用分层抽样按 8:2 划分为训练集与测试集，保证正负样本比例一致。分别训练逻辑回归、决策树与随机森林三类模型；在测试集上计算混淆矩阵、AUC 并绘制 ROC 曲线。所有实验在 Python 3.13 环境下使用 scikit-learn 完成。

（三）结果分析

表 1 汇总了两个数据集上三类模型的 AUC 表现。

模型	乳腺癌数据集 AUC	股票次日涨跌数据集 AUC
逻辑回归	0.9954	0.5408
决策树	0.9147	0.5635
随机森林	0.9932	0.5133

表 1 两类数据集上三类模型的 AUC 对比

由表 1 可见，在乳腺癌数据集上三类模型均取得较高 AUC，最优为逻辑回归（AUC = 0.9954），说明医学特征对良性/恶性具有强区分能力，随机森林与逻辑回归均明显优于决策树。在股票次日涨跌数据集上，最优模型为决策树（AUC = 0.5635），但整体 AUC 接近 0.5，说明仅凭日行情技术指标难以有效预测次日方向，市场噪声与有效市场假说在此得到印证。

图 1 与图 2 给出了乳腺癌数据集上的 ROC 曲线与混淆矩阵。ROC 曲线紧贴左上角，混淆矩阵中 TP 与 TN 占绝对多数，直观反映了模型的高区分度。

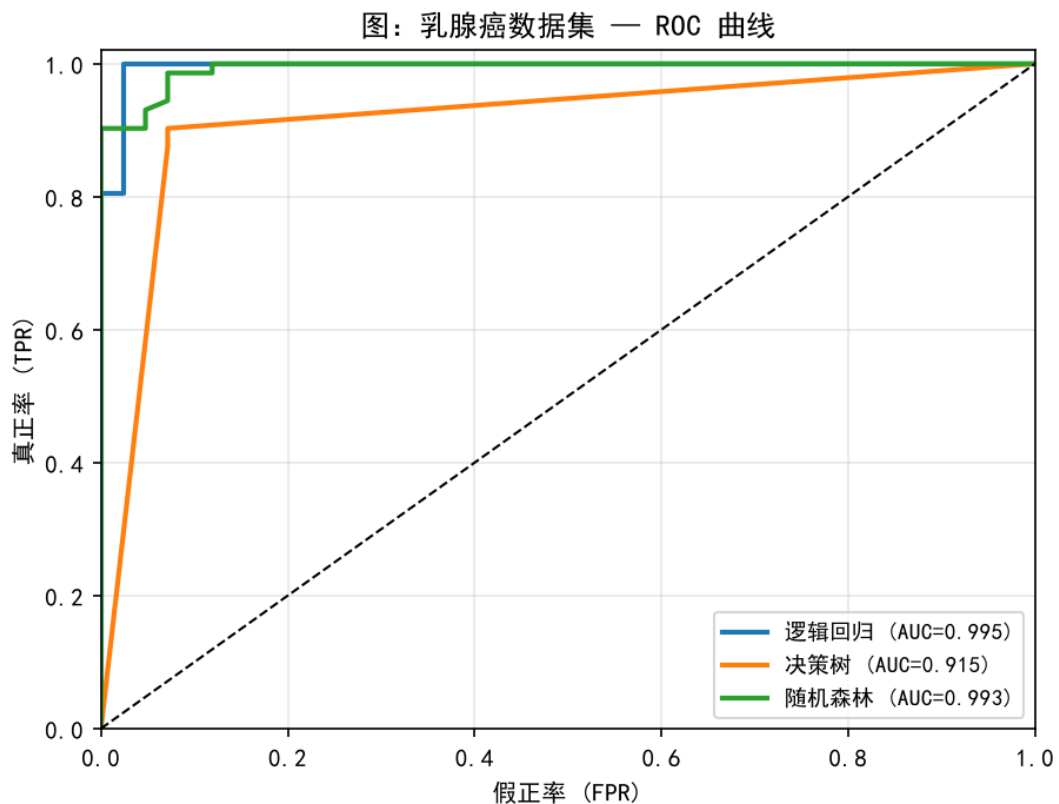


图 1 乳腺癌数据集 ROC 曲线

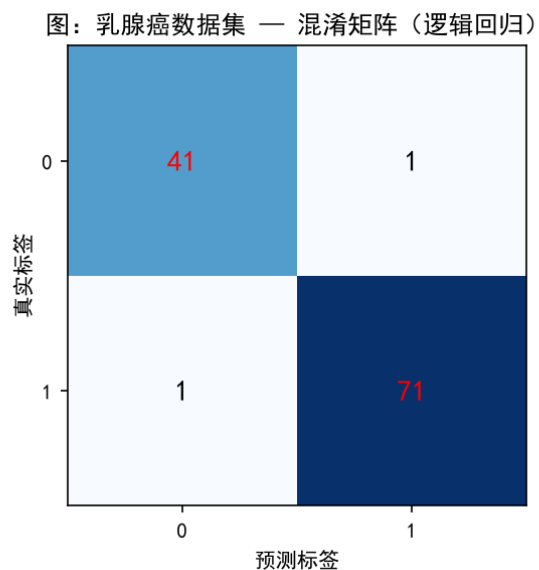


图 2 乳腺癌数据集混淆矩阵（逻辑回归）

图 3 与图 4 给出了股票次日涨跌数据集上的 ROC 曲线与混淆矩阵。ROC 曲线接近对角线，混淆矩阵中预测错误比例较高，与较低的 AUC 一致。

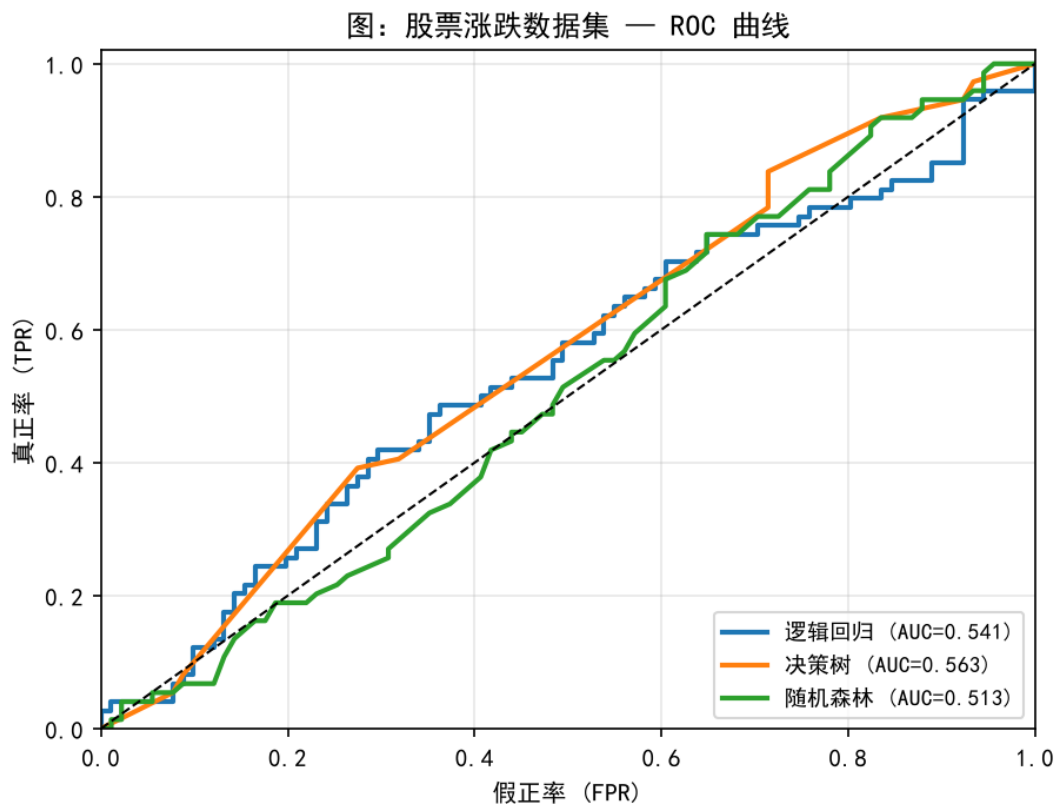


图 3 股票次日涨跌数据集 ROC 曲线

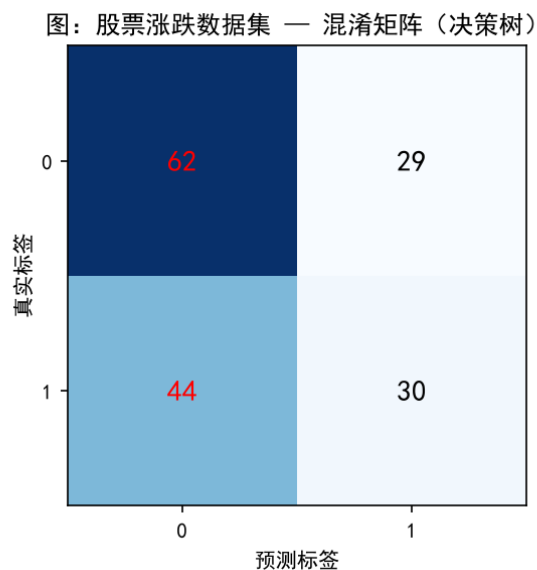


图 4 股票次日涨跌数据集混淆矩阵（决策树）

五、结论

本报告系统梳理了逻辑回归、决策树与随机森林三种分类算法的原理，并介绍了混淆矩阵、ROC 曲线与 AUC 等模型评价指标。实证部分分别以乳腺癌数据集和贵州茅台次日涨跌数据集为例，展示了完整的机器学习建模流程：数据加载、训练/测试集划分、模型训练、AUC 计算与 ROC 曲线绘制。

实验结果表明，在特征与标签关系明确的医学数据上，分类模型能够获得优异的预测性能；而在金融市场中，由于价格受多重噪声与随机冲击影响，单纯依赖技术指标的次日涨跌预测难以稳定跑赢随机水平。这提示在量化交易中应用机器学习时，需要引入更丰富的特征（如基本面、另类数据、文本情绪等）与更严谨的策略验证框架，以提升模型的实盘价值。

附录：Python 源代码

1. 数据构造与模型训练脚本（train_eval.py）

```
# -*- coding: utf-8 -*-
"""
train_eval.py
对两个二分类数据集（乳腺癌 / 股票涨跌）分别：
    1) 80/20 分层抽样划分训练集 / 测试集
    2) 训练 逻辑回归、决策树、随机森林 三个模型
    3) 计算 AUC、混淆矩阵、分类报告
    4) 绘制 ROC 曲线 + 混淆矩阵热力图，保存 PNG
结果写入 results.json / results.txt 供报告读取。
"""

import os
import json
import warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (roc_auc_score, roc_curve, confusion_matrix,
                             classification_report)

warnings.filterwarnings("ignore")

# ----- 中文字体 -----
FONT_PATH = r"C:\Windows\Fonts\simhei.ttf"
if os.path.exists(FONT_PATH):
    font_manager.fontManager.addfont(FONT_PATH)
    plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

FIG_DIR = "figures"
os.makedirs(FIG_DIR, exist_ok=True)

MODELS = {
    "逻辑回归": LogisticRegression(max_iter=1000, random_state=42),
    "决策树": DecisionTreeClassifier(max_depth=6, random_state=42),
    "随机森林": RandomForestClassifier(n_estimators=200, random_state=42),
}

COLORS = {"逻辑回归": "#1f77b4", "决策树": "#ff7f0e", "随机森林": "#2ca02c"}

def load_breast():
    d = load_breast_cancer()
    return d.data, d.target, "乳腺癌数据集"

def load_stock():
    df = pd.read_csv("stock_clf.csv")
    y = df["target"].values
    X = df.drop(columns=["target"]).values
    return X, y, "股票涨跌数据集"

def evaluate(X, y, title):
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)

    scaler = StandardScaler().fit(X_tr)
    X_tr_s = scaler.transform(X_tr)
    X_te_s = scaler.transform(X_te)
```

```

res = {"n_train": int(len(y_tr)), "n_test": int(len(y_te)),
      "auc": {}, "cm": {}, "report": {}}
roc_data = {}

for name, model in MODELS.items():
    model.fit(X_tr_s, y_tr)
    proba = model.predict_proba(X_te_s)[: , 1]
    pred = (proba >= 0.5).astype(int)

    auc = roc_auc_score(y_te, proba)
    fpr, tpr, _ = roc_curve(y_te, proba)
    cm = confusion_matrix(y_te, pred)

    res["auc"][name] = round(float(auc), 4)
    res["cm"][name] = cm.tolist()
    res["report"][name] = classification_report(
        y_te, pred, output_dict=True, zero_division=0)
    roc_data[name] = (fpr, tpr, auc)

# ---- ROC 曲线图 ----
plt.figure(figsize=(6.2, 4.8))
for name, (fpr, tpr, auc) in roc_data.items():
    plt.plot(fpr, tpr, color=COLORS[name], lw=2,
             label=f"{name} (AUC={auc:.3f})")
plt.plot([0, 1], [0, 1], "k--", lw=1)
plt.xlim(0, 1); plt.ylim(0, 1.02)
plt.xlabel("假正率 (FPR)"); plt.ylabel("真正率 (TPR)")
plt.title(f"图: {title} — ROC 曲线")
plt.legend(loc="lower right", fontsize=9)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(f"{FIG_DIR}/roc_{title[:4]}.png", dpi=150)
plt.close()

# ---- 混淆矩阵热力图 (取 AUC 最高的模型) ----
best = max(res["auc"], key=lambda k: res["auc"][k])
cm = np.array(res["cm"][best])
plt.figure(figsize=(4.6, 4.0))
plt.imshow(cm, cmap="Blues")
plt.title(f"图: {title} — 混淆矩阵 ({best})")
plt.xlabel("预测标签"); plt.ylabel("真实标签")
plt.xticks([0, 1], ["0", "1"]); plt.yticks([0, 1], ["0", "1"])
for i in range(2):
    for j in range(2):
        plt.text(j, i, str(cm[i, j]), ha="center", va="center",
                 color="red" if cm[i, j] > cm.max() / 2 else "black",
                 fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig(f"{FIG_DIR}/cm_{title[:4]}.png", dpi=150)
plt.close()

res["best_model"] = best
return res

def main():
    all_res = {}
    for loader, key in [(load_breast, "breast"), (load_stock, "stock")]:
        X, y, title = loader()
        print(f"\n===== {title} ===== 样本数={len(y)} 正类占比={y.mean():.3f}")
        all_res[key] = evaluate(X, y, title)
        for m, a in all_res[key]["auc"].items():
            print(f" {m:<8} AUC={a:.4f} best={m == all_res[key]['best_model']}")

    with open("results.json", "w", encoding="utf-8") as f:
        json.dump(all_res, f, ensure_ascii=False, indent=2)

    with open("results.txt", "w", encoding="utf-8") as f:
        f.write("TASK5 模型评估结果\n")
        for key, r in all_res.items():
            f.write(f"\n[{key}] 训练集={r['n_train']} 测试集={r['n_test']}\n")
            for m, a in r["auc"].items():

```



```
f.write(f" {m}: AUC={a} 混淆矩阵={r['cm'][m]}\n")

print("\n结果已写入 results.json / results.txt")

if __name__ == "__main__":
    main()
```